

Python : Orienté objets

Tables des matières

- | | |
|----------------------|-----------------------|
| 1. Introduction | 7. Polymorphisme |
| 2. Structures | 8. Classes abstraites |
| 3. Classes et objets | 9. Membres statiques |
| 4. Encapsulation | 10. Interfaces |
| 5. Data model | 11. Constructeurs |
| 6. Héritage | |

1. Introduction

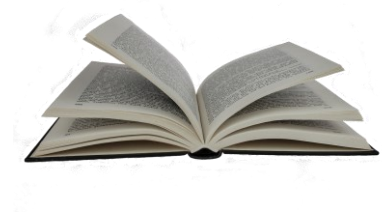
- Définition de la programmation orientée objet
- Comparaison avec la programmation procédurale
- Principes fondamentaux
- Historique et évolution
- Importance de la POO

Définition de la programmation orientée objet

La POO offre une approche de programmation qui repose sur la modélisation des entités du monde réel sous forme d'objets, chacun possédant des caractéristiques (**attributs**) et des actions (**méthodes**) qui lui sont propres.



Cette approche permet de structurer le code de manière logique et intuitive, en favorisant la réutilisabilité, la modularité et la maintenance aisée du code. En permettant de représenter des entités complexes sous forme d'objets autonomes interagissant entre eux.



La POO permet à un objet de prendre pratiquement n'importe quelle forme, ce qui rend la modélisation des concepts du monde réel ou fictif plus flexible et puissante.



Comparaison avec la programmation procédurale

Contrairement à la programmation procédurale, qui divise le code en séquences d'instructions, la POO organise le code autour de données et de fonctions qui agissent sur ces données, rassemblées au sein d'objets.

Cette approche permet une meilleure modélisation des concepts du monde réel, en favorisant la compréhension du code et en réduisant les risques d'erreurs.

La POO encourage la réutilisation du code grâce à la création de bibliothèques d'objets réutilisables, ce qui accélère le processus de développement et améliore la qualité du code produit.

Procédurale :

```
1 def creer_chat(nom, couleur):
2     return {"nom": nom, "couleur": couleur}
3
4 def miauler(chat):
5     return f"{chat['nom']} miaule."
6
7 # Création d'un chat et appel de la fonction miauler
8 mon_chat = creer_chat("Whiskers", "gris")
9 print(miauler(mon_chat)) # Affiche : Whiskers miaule.
```

POO :

```
1 class Chat:
2     def __init__(self, nom, couleur):
3         self.nom = nom
4         self.couleur = couleur
5
6     def miauler(self):
7         return f"{self.nom} miaule."
8
9 # Création d'un objet chat
10 mon_chat = Chat("Whiskers", "gris")
11 print(mon_chat.miauler()) # Affiche : Whiskers miaule.
```

Principes fondamentaux

Outre les concepts d'encapsulation, d'héritage et de polymorphisme qui seront vu dans les chapitre suivants, la POO repose sur le principe d'abstraction, qui consiste à isoler les aspects essentiels d'un objet tout en cachant les détails d'implémentation superflus.

Cette abstraction permet de simplifier la complexité du système en fournissant des interfaces claires et cohérentes pour interagir avec les objets.

En combinant ces principes, la POO offre un cadre flexible et extensible pour la conception de logiciels complexes, facilitant la maintenance et l'évolution du code au fil du temps.



Historiques et évolution de la POO

La POO trouve ses origines dans les travaux pionniers réalisés dans les années 1960 avec le langage de programmation Simula, qui a introduit les concepts de classes et d'objets.

Depuis lors, la POO a connu un développement rapide avec l'émergence de langages tels que Smalltalk, C++, Java et Python, qui ont étendu et popularisé les concepts de la POO. Aujourd'hui, celle-ci est largement utilisée dans le développement logiciel pour sa capacité à modéliser des systèmes complexes comme le montre le tableau suivant.

Année	Événement
1960	Simula, le premier langage de programmation à introduire les concepts de classes et d'objets, est développé.
Années 1970	Smalltalk émerge comme un langage de programmation influent popularisant les idées de la POO, y compris les classes, les objets et l'encapsulation.
Années 1980	C++, un langage de programmation hybride combinant la programmation procédurale avec des concepts de la POO comme l'héritage et le polymorphisme, est créé.

Historiques et évolution de la POO

Années 1990	Java est lancé, un langage de programmation orienté objet conçu pour être portable et sûr, contribuant à la popularisation de la POO.
Années 2000	Python gagne en popularité comme un langage de programmation polyvalent avec un support intégré pour la POO, offrant une syntaxe propre et une grande flexibilité.
Années 2010	L'importance de la POO dans l'ingénierie logicielle moderne est encore renforcée par l'essor des méthodologies de développement agile et des pratiques de développement centrées sur les objets.
Années 2020	La POO continue d'évoluer avec l'émergence de nouvelles tendances telles que la programmation orientée objet fonctionnelle et les architectures logicielles basées sur les microservices.
2024	Les principes de la POO demeurent fondamentaux dans la conception et le développement de logiciels, contribuant à la création de systèmes évolutifs, flexibles et maintenables.

Importance de la POO

Au-delà de ses avantages techniques, la POO favorise des pratiques de développement plus collaboratives et agiles, en encourageant la modularité, la réutilisabilité et la maintenabilité du code.

En permettant de représenter des entités du monde réel de manière naturelle, la POO facilite la communication entre les membres de l'équipe de développement et favorise une meilleure compréhension des exigences du projet.

Son impact s'étend également au-delà du développement logiciel, influençant des domaines tels que l'analyse des données, l'intelligence artificielle et l'ingénierie logicielle.

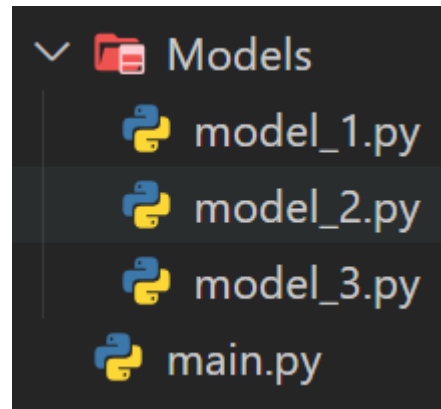
2. Structures

- Travailler avec plusieurs fichiers
- Logique d'imports
- Limite d'exécution
- Bonnes pratiques

Travailler avec plusieurs fichiers

Il est important de structurer le code sur plusieurs fichiers afin de séparer les différentes classes et fonctionnalités. Chaque fichier peut contenir des classes et des méthodes associées à des aspects spécifiques du projet.

Cette approche favorise la modularité et la réutilisabilité du code, en nous permettant de se concentrer sur des parties spécifiques de l'application.



Logique d'import

L'instruction `from Models.model_1 import model_1` est une façon de charger spécifiquement une classe ou une fonction appelée `model_1` depuis un fichier Python appelé `model_1.py` situé dans le répertoire `Models`.

Cette méthode d'importation permet d'accéder directement à la classe ou à la fonction sans avoir besoin de spécifier le nom du module lors de son utilisation. Ensuite, en utilisant `test = model_1()`, nous créons une instance de la classe `model_1`, prête à être utilisée dans notre programme.

Ce type d'importation sélective est couramment utilisé dans les projets pour simplifier l'accès à des fonctionnalités spécifiques et rendre le code plus lisible et modulaire.

```
1  from Models.model_1 import model_1
2  test = model_1()
```

Limite d'exécution

Lors de l'importation d'un fichier Python, il est souvent nécessaire de restreindre l'exécution du code contenu dans ce fichier.

Cela peut être réalisé en utilisant le nom du scope, accessible via la variable spéciale `__name__`. Lorsque le fichier est exécuté directement en tant que point d'entrée, la valeur de `__name__` est définie sur "main".

Ainsi, en vérifiant si `__name__` est égal à "main", nous pouvons conditionner l'exécution de certaines parties du code à cet état.

```
1  def fonction_principale():
2      print("Code exécuté uniquement si le fichier est le point d'entrée")
3
4  if __name__ == "__main__":
5      fonction_principale()
```

Bonnes pratiques

Pour maintenir une structure de projet cohérente et efficace, il est recommandé d'adopter certaines bonnes pratiques :

- **Utiliser des noms descriptifs** : Choisissez des noms de classe qui reflètent clairement le rôle ou la fonction de la classe dans votre système. Les noms descriptifs facilitent la compréhension du code par d'autres développeurs.
- **Éviter les abréviations cryptiques** : Évitez les abréviations peu claires ou cryptiques dans les noms de classe. Optez plutôt pour des noms complets et explicites qui rendent le code plus compréhensible.
- **Respecter la convention de nommage PascalCase** : Suivez la convention de nommage **PascalCase** pour les noms de classe. Cela contribue à maintenir la cohérence dans le code Python et facilite la collaboration.
- **Un seul concept par classe** : Concevez vos classes de manière à ce qu'elles aient une seule responsabilité ou un seul concept. Cela favorise la modularité et la réutilisabilité du code.
- **Documenter les classes** : Utilisez des **docstrings** pour documenter vos classes, en fournissant des descriptions claires de leur fonctionnement, de leurs attributs et de leurs méthodes. Une documentation claire facilite l'utilisation et la maintenance du code.

3. Classe et objets

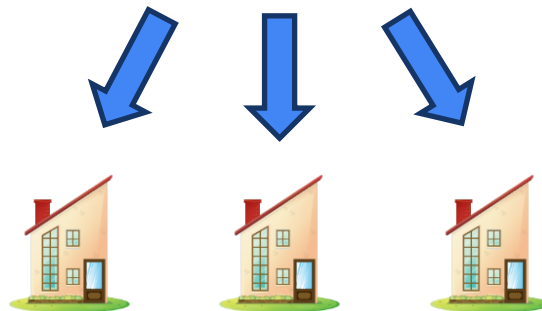
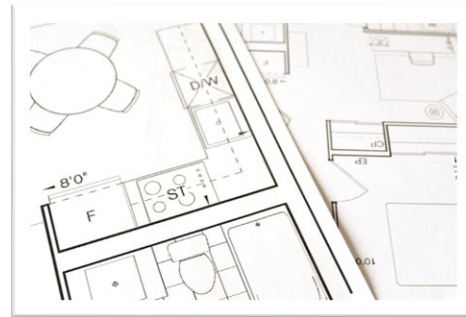
- Définition d'une classe
- Syntaxe de déclaration de classe
- Création et instanciation d'objets
- Attributs
- Méthodes

Définition d'une classe

Les classes en Python servent de modèles pour créer des objets ayant des **attributs** (états) et des **méthodes** (comportements) spécifiques.

Elles agissent comme des plans de construction ou des moules, définissant la structure et le comportement des objets qu'ellesinstancient.

Chaque objet créé à partir d'une classe est une instance de cette classe, permettant de créer autant d'objets que nécessaire avec les mêmes attributs et méthodes définis dans la classe.



Syntaxe de déclaration de classe

La syntaxe de déclaration de classe en Python est simple. Elle commence par le mot-clé `class`, suivi du nom de la classe que nous souhaiterons créer. Ce mot-clé est fondamental car il indique à Python que nous sommes sur le point de définir une nouvelle classe d'objets.

Dans cet exemple, nous avons utilisé le mot-clé `class` pour déclarer une classe nommée `Voiture`. Ceci crée une structure de classe vide, prête à être remplie avec des attributs et des méthodes spécifiques à la classe.

En utilisant la syntaxe de classe, nous pourrions définir des modèles de données et des fonctionnalités qui seront utilisés pour créer des objets de cette classe.

```
1 class Voiture:  
2     pass
```

Création et instantiation des objets

L'étape après la déclaration d'une classe est la création et l'instantiation d'objets de cette classe. Pour créer un objet à partir d'une classe en Python, nous utiliserons simplement le nom de la classe suivi de parenthèses. Cette opération est appelée instantiation.

Dans cet exemple, nous créons un nouvel objet de la classe Voiture en utilisant la syntaxe `Voiture()`. Cela crée une instance de la classe Voiture, que nous stockons dans la variable `voiture1`. L'objet `voiture1` est maintenant une instance de la classe Voiture et peut être utilisé pour accéder aux attributs et aux méthodes définis dans la classe.

```
1  # Création d'un objet de la classe Voiture
2  from Voiture import Voiture
3
4
5  voiture1 = Voiture()
```

Attributs

Les attributs représentent les caractéristiques ou les propriétés d'un objet.

Dans le contexte d'une **voiture**, les attributs pourraient inclure des informations telles que la **marque**, le **modèle**, la **couleur**, la **vitesse actuelle**, et le **nombre de portes**.

Chaque **attributs** joue un rôle essentiel dans la définition de l'état de l'objet et peut être consulté ou modifié par les méthodes de la classe associée ou par ces **attributs** en l'absence de **propriétés** qui les contrôlent.

```
1 class Voiture:
2     marque = None
3     modele = "Corolla"
4     couleur = None
5     vitesse_actuelle = 0
6
7     # Création d'une instance de voiture
8     ma_voiture = Voiture()
9
10    # Affichage des attributs de la voiture
11    print("Marque :", ma_voiture.marque)
12    print("Modèle :", ma_voiture.modele)
13    print("Couleur :", ma_voiture.couleur)
14    print("Vitesse actuelle :", ma_voiture.vitesse_actuelle)
```

Méthodes

les **méthodes** jouent un rôle essentiel en permettant aux objets d'effectuer des actions ou d'interagir avec d'autres objets. Elles représentent les comportements ou les fonctionnalités associées à un objet particulier.

Celles-ci pourraient inclure des actions telles que l'accélération, le freinage ou le calcul de la consommation de carburant.

Elles encapsulent le comportement de l'objet, offrant ainsi une interface cohérente pour manipuler et interagir avec lui.

```
1 class Voiture:
2     marque = None
3     modele = "Corolla"
4     couleur = None
5     vitesse_actuelle = 0
6
7     def accelerer(self, acceleration):
8         self.vitesse_actuelle += acceleration
9
10    def freiner(self, deceleration):
11        self.vitesse_actuelle -= deceleration
12
13    def afficher_info(self):
14        print("Marque :", self.marque if self.marque is not None else "Non définis !")
15        print("Modèle :", self.modele)
16        print("Couleur :", self.couleur if self.couleur is not None else "Non définis !")
17        print("Vitesse actuelle :", self.vitesse_actuelle)
18
19    # Création d'une instance de voiture
20    ma_voiture = Voiture()
21
22    # Affichage des informations initiales de la voiture
23    print("Informations initiales de la voiture :")
24    ma_voiture.afficher_info()
25
26    # Accélération de la voiture
27    ma_voiture.accelerer(20)
28    print("\nNouvelle vitesse après accélération :")
29    ma_voiture.afficher_info()
```

Exercices

1. Créer une class Elephant contenant :

Caractéristiques :

- nom
- appétit (sur 100)
- satisfaction (sur 100)
- en_vie
- soigneur

Comportement :

- manger()

Exercices

2. Créer une class soigneur contenant :

Caractéristiques :

- nom
- date_naissance
- expérience
- nombre_animaux_responsable

Comportement (si le soigneur est responsable de l'animal) :

- nourrir(animal)
- entretenir(animal)

Exercices

3. Créer une class enclos contenant :

Caractéristiques :

- nom
- capacite_max
- taille
- liste_animaux

Comportement :

- ajouter_animal(animal)
- enleve_animal(animal)
- afficher_animaux()
- Bonus : Imaginer un comportement permettant de simuler un passage de jour.